Event-Driven Workflow Management System

Yermek Aubayev — Matricola 551098

21 August 2026

Executive summary

This individual Software Engineering project implements a bounded Workflow Management System for a small organization. Purchase Request Approval is the reference workflow: a requester provides structured business data, the system validates it, creates persistent approval work, waits without holding a request thread, resumes the same run after a decision or restart, creates an internal Purchase Authorization after approval, and records an ordered audit history. Authorization does not place an order or transfer money.

The professor approved the Event-Driven Workflow Management System direction and accepted orchestration and choreography as the two complex functionalities. Purchase Request Approval is the final reference scenario selected to make that approved objective understandable; no claim is made that every scenario or architectural decision was separately approved.

Problem, users, and useful result

Small organizations often handle repeatable approvals through messages and spreadsheets. Status, ownership, retries, and decision history become unclear. The requester needs a visible outcome; the approver needs persistent contextual work; the process owner needs safe configuration; the evaluator needs inspectable evidence.

The useful result is an authoritative state: validation failure, waiting, rejected, internally authorized, or manual action required, accompanied by a notification and ordered trace.

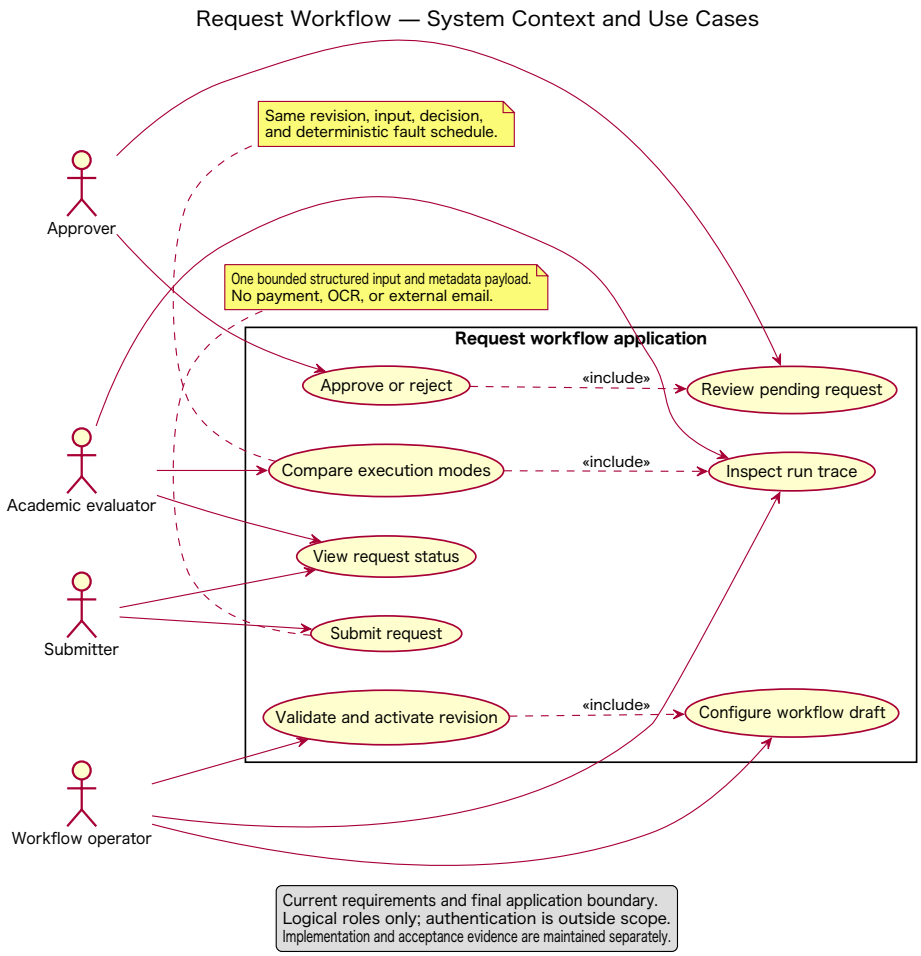
Requirements and reference workflow

The structured request contains requester, department, item/service, supplier, amount, currency, justification, and required date. Validation requires non-empty bounded text, positive amount with at most two decimals, uppercase currency, a valid non-past required date, and at least 20 meaningful justification characters.

The active four-step workflow is:

1. Validate Purchase Request;
2. Human Approval;
3. Create Purchase Authorization;
4. Record Notification.

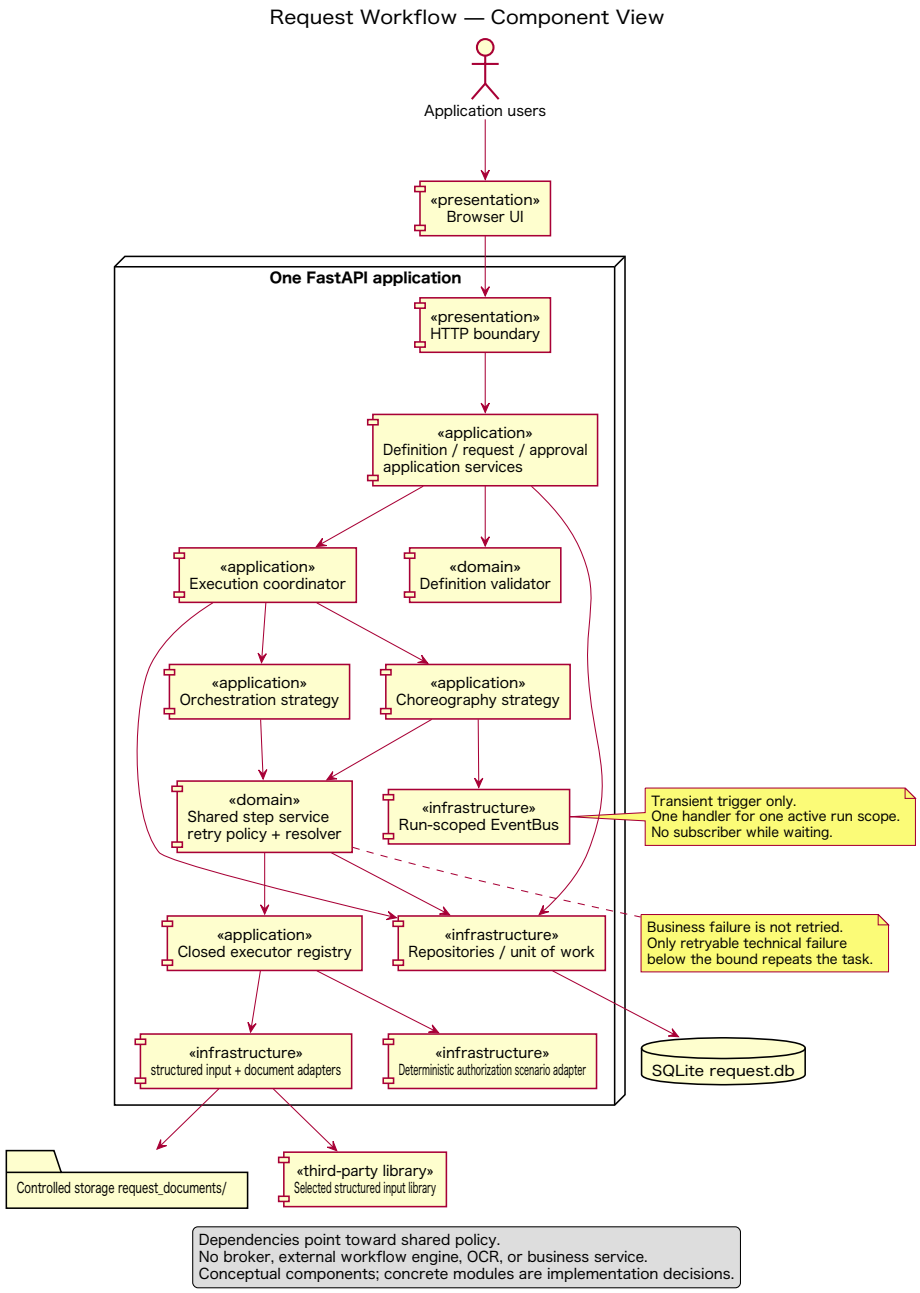
The acceptance scenarios are approval, rejection, invalid input, one temporary authorization failure followed by success, and retry exhaustion leading to manual action.



System context and use cases

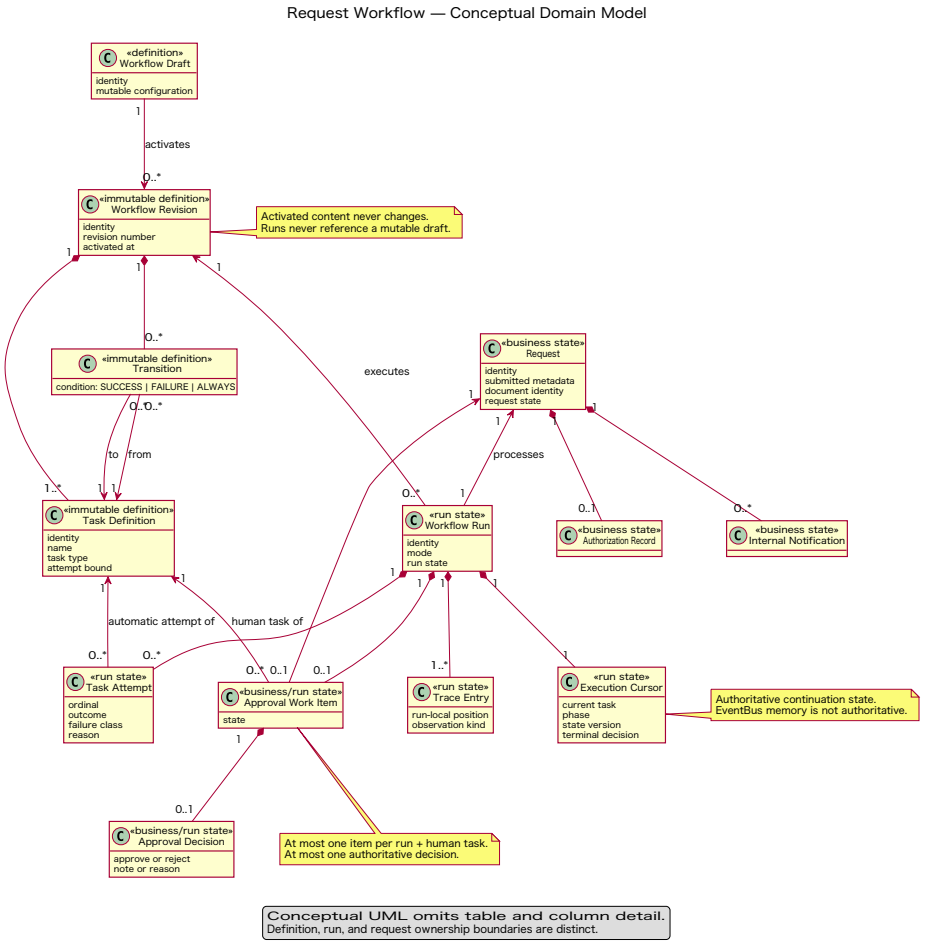
Architecture

The architecture is a layered modular monolith: one FastAPI process, one SQLite database, and an in-process synchronous EventBus. There are no brokers, microservices, external workflow engines, payment, accounting, ERP, supplier, or email integrations.

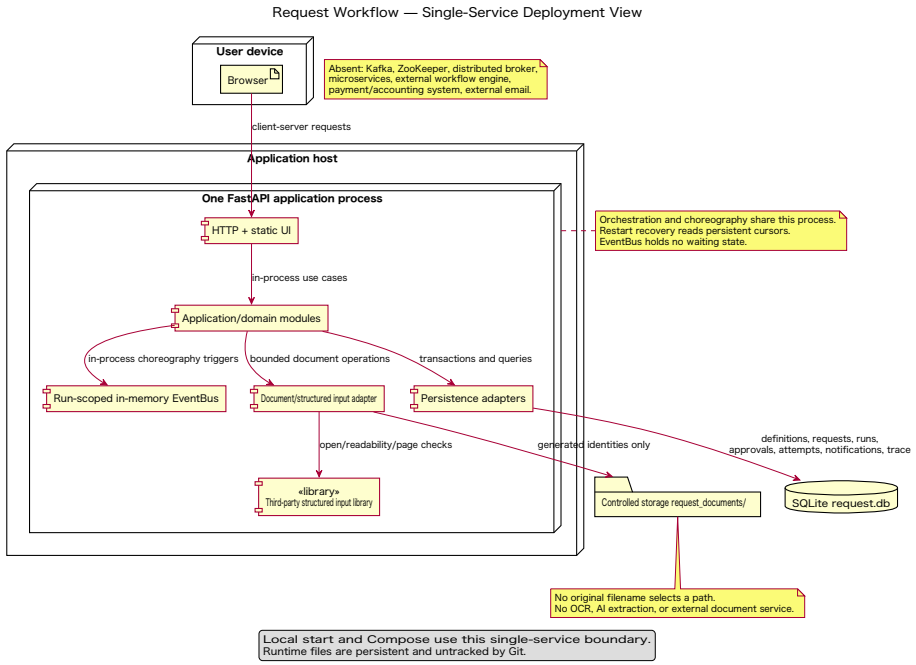


Component view

Workflow definitions become immutable revisions. Each run references one revision and owns its cursor, attempts, approval, effects, and audit entries.



Domain model



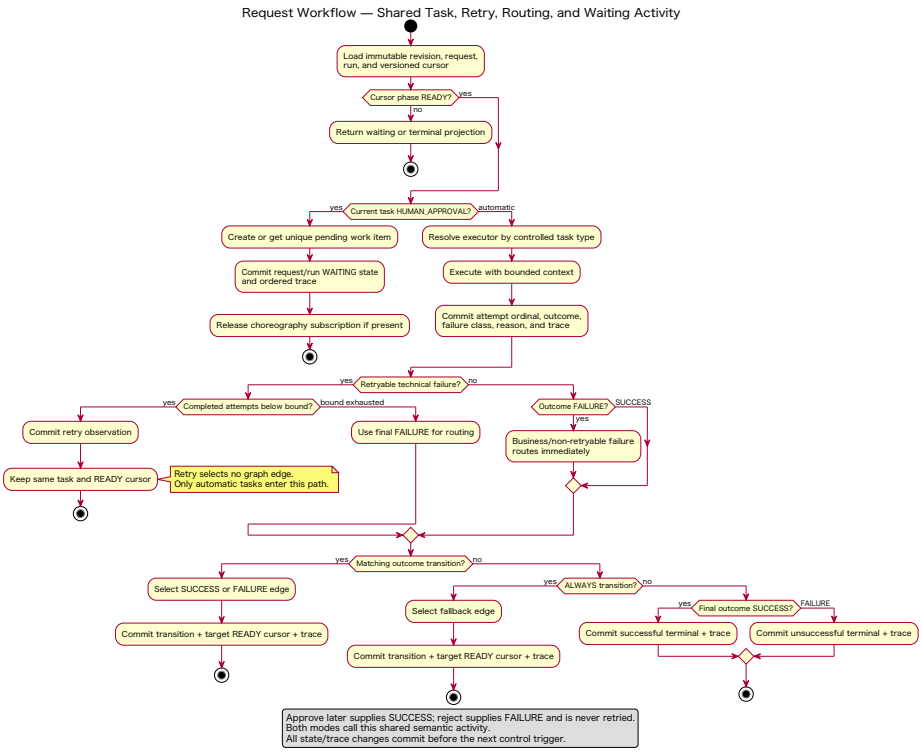
Deployment view

Workflow Designer

The designer is intentionally bounded. A process owner may configure task keys/names, one start, the four safe task types, SUCCESS/FAILURE/ALWAYS transitions, and positive attempt bounds for automatic tasks. Graph validation rejects missing or multiple starts, unknown references/types, unreachable tasks, cycles, ambiguous equal-precedence transitions, invalid terminals, and retry on human approval. No scripts, expressions, plugins, fork/join, nested workflows, or arbitrary code are accepted.

Custom logic

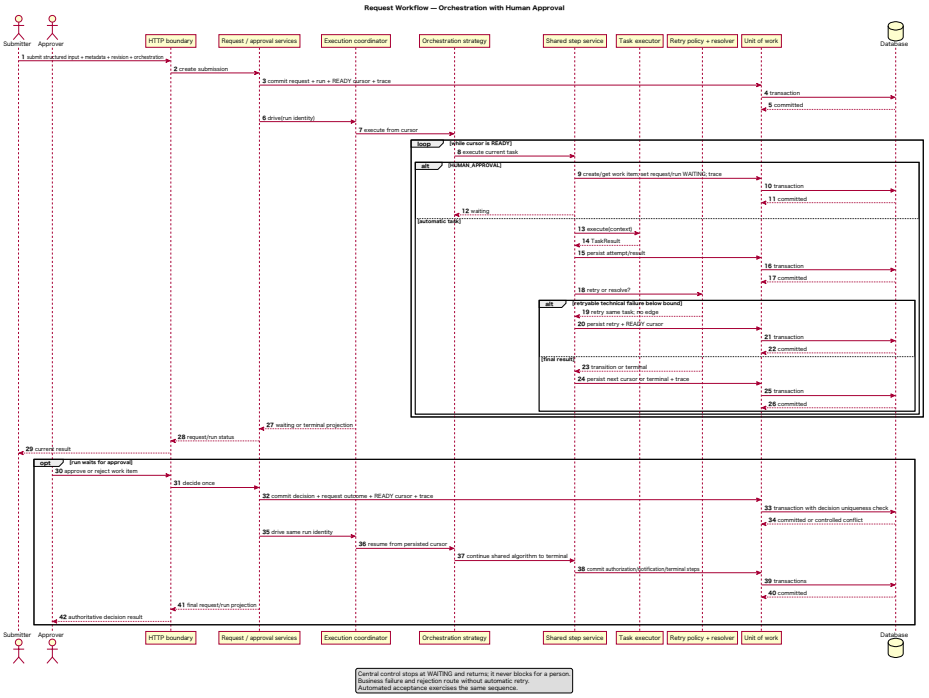
The student-implemented logic comprises complete DAG validation, deterministic transition selection, classified bounded retry before routing, atomic cursor/effect/trace commits, persistent wait/resume, idempotent decision handling, immutable revision consistency, run isolation, centralized orchestration, and run-scoped event-reaction choreography.



Retry, routing, and waiting activity

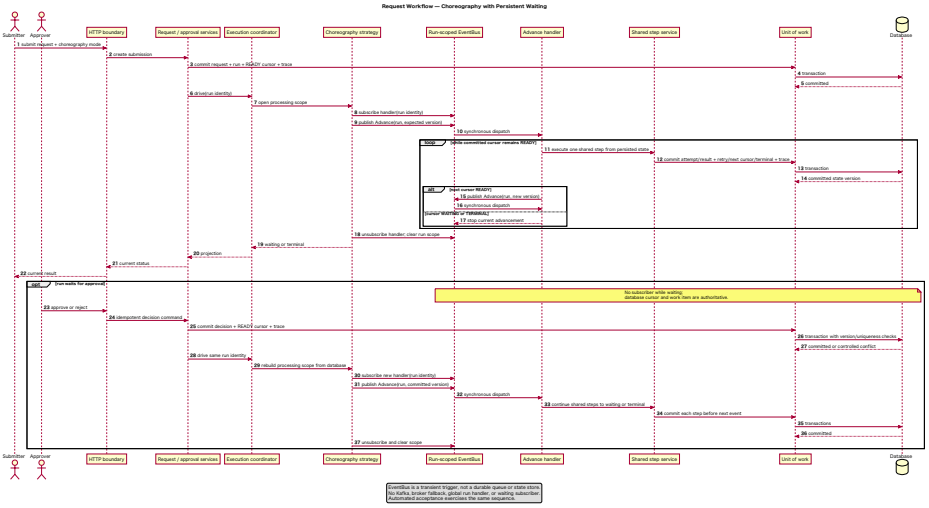
Orchestration and choreography

Orchestration uses one central loop that reads the committed cursor and invokes the shared step service until waiting or terminal state.



Orchestration sequence

Choreography temporarily subscribes one handler for one run. `AdvanceRun` events synchronously trigger the next committed step; the handler is removed at waiting, terminal state, or error. The database cursor is authoritative and the EventBus is not durable.

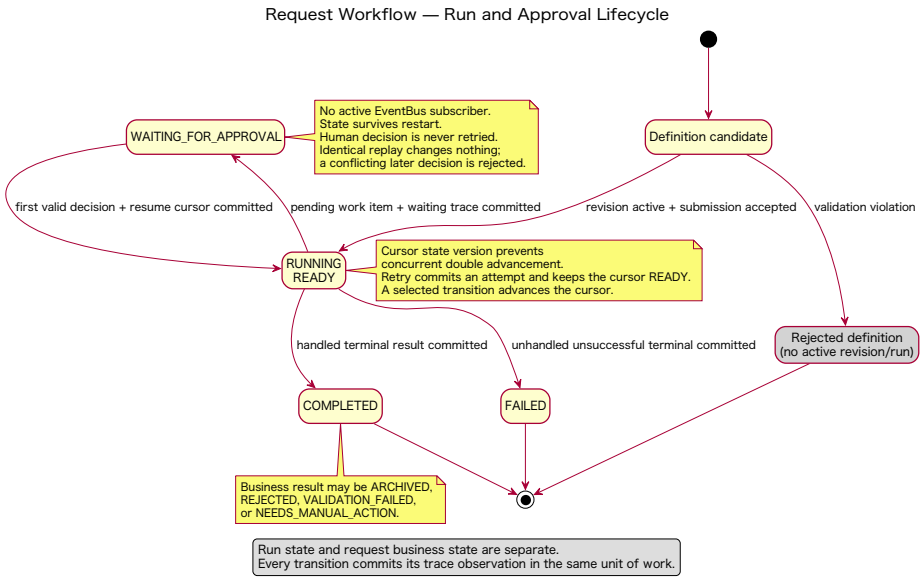


Choreography sequence

This supports an academic comparison of control placement under identical business semantics. It does not support claims of distributed choreography, asynchronous delivery, exactly-once messaging, or independent services.

Persistence and recovery

SQLite stores drafts, immutable revisions, requests, runs, cursors, attempts, work items, decisions, Purchase Authorizations, notifications, and trace entries. A restart test starts Uvicorn, creates a waiting run, stops the process, starts a new process against the same isolated database, resumes the same run, and verifies authorization.



Run lifecycle

Verification

The final repository run reports **98 passing tests**. Tests cover domain validation, graph rules, resolver precedence, retry bounds, constructor activation, orchestration, choreography, wait/resume, approval/rejection, replay/conflict, run isolation, failure scenarios, API journeys, restart, and deployment inventory. Static string checks are supporting evidence, not business acceptance evidence.

Development and evolution

The work fits an approximately 100-hour individual scope by using one reference workflow, four safe task types, one process, and no external integration. An intermediate Invoice Approval reference application was superseded after evaluation showed that file processing distracted from the workflow-management objective. The graph, retry, coordination, persistence, and audit core was preserved while the final reference workflow became structured Purchase Request Approval.

Reuse

FastAPI, Uvicorn, SQLAlchemy, Pydantic, pytest, HTTPX, SQLite, Docker, and PlantUML provide general-purpose infrastructure. They do not supply the workflow semantics, graph validator, resolver, retry policy, persistence ownership, approval lifecycle, coordination strategies, or acceptance scenarios.

Limitations

This is an academic prototype. It has no authentication, role authorization, external procurement integration, order placement, payment, accounting, email, durable event broker, horizontal scaling, high availability, or production security/operations hardening. Demonstration failures are explicit deterministic examination controls, not random production behavior.

Conclusion

The final product makes repeatable approvals understandable and auditable. A structured request reaches a persistent human decision, survives waiting and restart, produces an internal authorization or controlled negative outcome, and retains evidence. The bounded Workflow Designer demonstrates reuse, while orchestration and run-scoped synchronous choreography expose two control strategies over the same rules.